

Java API Collections

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

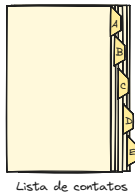
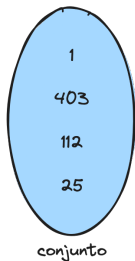
Motivação

- Suponha que você precise armazenar uma lista de nomes de pessoas. Nesta lista, você pode querer adicionar, remover ou pesquisar nomes.
- O uso de um vetor seria uma solução simples, como no exemplo abaixo:

```
String [] nomes = new String[1000];  
nomes[0] = "João";  
nomes[1] = "Maria";  
// ...
```

- Seria fácil adicionar novos nomes, mas e se você quisesse saber quantos nomes foram armazenados? Ou se não quisesse armazenar nomes duplicados? Ou ainda, se quisesse remover um nome específico?

Grupos naturais de objetos



Conjunto de números naturais

- Conjunto não ordenado de elementos
- Não permite elementos duplicados

Lista de contatos

- Permite elementos duplicados
- Oferece uma forma rápida de acessar elementos

Coleção em Java

- **Coleção é um objeto** que **representa um conjunto de objetos** dentro de uma única unidade
 - Permite armazenar, obter e manipular dados agregados com facilidade
- Coleção é um **tipo de dado abstrato** que representa um conjunto de objetos
 - baralho
 - pasta de e-mails
 - catálogo telefônico
 - ...

Java Collections Framework I

<https://docs.oracle.com/en/java/javase/21/core/java-collections-framework.html>

Benefícios do uso do *Java Collections Framework*

- Redução de tempo de desenvolvimento
- Redução de complexidade do código
- Aumentar a eficiência e qualidade código
- Aumentar a portabilidade do código

Java Collections Framework II

<https://docs.oracle.com/en/java/javase/21/core/java-collections-framework.html>

■ Interfaces

- Tipos de dados abstratos para representar coleções de objetos
- Ex: List, Set, Queue, Map

■ Implementações

- Estruturas de dados para armazenar e manipular grupos de objetos
- Ex: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap

Java Collections Framework III

<https://docs.oracle.com/en/java/javase/21/core/java-collections-framework.html>

■ Algoritmos

- Operações comuns para manipular coleções de objetos
- Ex: Busca, ordenação, embaralhamento, agrupamento, etc.

■ Classes utilitárias

- `Collections` – Métodos estáticos para operações comuns em coleções
- `Arrays` – Métodos estáticos para operações comuns em vetores
- `Comparator` – Interface funcional para ordenação de objetos

Java Collections Framework I

Interfaces e suas implementações

■ List

- Coleção ordenada e permite elementos duplicados, sendo possível acessar elementos por índice
- Implementações: `ArrayList`, `LinkedList`, `Vector`, `Stack`

■ Set

- Segue a abstração de conjuntos matemáticos e não permite elementos duplicados e não mantém a ordem de inserção
- Implementações: `HashSet`, `LinkedHashSet`, `TreeSet`

Java Collections Framework II

Interfaces e suas implementações

■ Queue

- Fila que ordena elementos para serem processados posteriormente, por exemplo, FIFO (primeiro que entra é o primeiro que sai)
- Implementações: `LinkedList`, `ArrayDeque`, `PriorityQueue`

■ Map

- Mapeia chaves para valores (também conhecido como dicionário)
- Não permite chaves duplicadas e cada chave mapeia somente um valor
- Implementações: `HashMap`, `LinkedHashMap`, `TreeMap`

Qual coleção usar?

Você precisa de uma coleção...

- que mantenha a ordem de inserção e permita elementos duplicados?
- que permita acessar elementos por índice?
- que não permita elementos duplicados?
- que permita armazenar nulos?
- que ofereça operações de busca, inserção e remoção eficientes?
- que opere em ambientes concorrentes? (*multi-thread*)

Exemplos de uso



Os exemplos a seguir são simplificados e visam apenas apresentar a sintaxe básica de uso das coleções. Como ainda não foi apresentado o conceito de herança e polimorfismo na disciplina, optou-se na pela declaração de variáveis utilizando classes concretas ao invés de interfaces.

ArrayList

Armazena objetos em um vetor, cujo tamanho aumenta automaticamente

```
// Criando uma lista de String
ArrayList<String> lista = new ArrayList<>(); // poderia ser de qualquer tipo, ex: Pessoa

// Adicionando elementos
lista.add("POO");
lista.add("IFSC");

// Obtendo o total de elementos na coleção
int total = lista.size();

// Convertendo para vetor
String[] v = (String[]) lista.toArray();

// Obtém elemento na posição 1
String nome = lista.get(1);

// Obtém elemento na última posição
String ultimo = lista.getLast();
```

ArrayList

Ordenação e embaralhamento de elementos

```
ArrayList<Integer> lista = new ArrayList<>();  
lista.add(2);  
lista.add(4);  
lista.add(1);  
  
// Ordenando com java.util.Comparator  
lista.sort(Comparator.naturalOrder());  
  
// Ordenando com Collections  
Collections.sort(lista);  
  
// Embaralhando elementos  
Collections.shuffle(lista);  
  
// Verifica se a lista está vazia  
boolean vazia = lista.isEmpty();
```

ArrayList

Percorrendo elementos de uma coleção

```
ArrayList<Pessoa> pessoas = new ArrayList<>();

// Adicionando objetos do tipo Pessoa na lista
pessoas.add(new Pessoa("Ana", 40));
pessoas.add(new Pessoa("Juca", 30));
pessoas.add(new Pessoa("Maria", 25));

// Usando for-each para percorrer a lista
for(Pessoa p: pessoas){
    System.out.println(p);
}

// Usando forEach com lambda
pessoas.forEach(pessoa -> System.out.println(pessoa));

// Usando forEach com referência de método
pessoas.forEach(System.out::println);
```


ArrayList

Uso de Streams¹ para manipular elementos

```
// Busca por um elemento cujo nome seja "Maria"
Pessoa p = pessoas.stream()
    .filter(pessoa -> pessoa.getNome().equals("Maria"))
    .findFirst()
    .orElse(null); // retorna null se não encontrar

if (p != null){ // se encontrou
    System.out.println(p);
}

// Ordenando os elementos da lista por idade
pessoas.sort(Comparator.comparing(Pessoa::getIdade));

// Exibindo todos os elementos da lista que atendem a uma condição, idade > 30
pessoas.stream()
    .filter(pessoa -> pessoa.getIdade() > 30)
    .forEach(System.out::println);
```

¹<https://www.oracle.com/br/technical-resources/articles/java-stream-api.html>

ArrayList

Removendo elementos

```
// Removendo elemento caso atenda a uma condição
// Exemplo: Remover todos os elementos cujo nome seja "Juca"
pessoas.removeIf(elemento->elemento.getNome().equals("Juca"));

// Removendo um elemento se o nome for "Ana" e a idade for 40
// Neste caso, o método remove() irá comparar o objeto passado como parâmetro com os
// objetos armazenados na coleção, utilizando o método equals da classe Pessoa para
// verificar se são iguais
pessoas.remove(new Pessoa("Ana", 40));
```

Nota

A classe Pessoa deve implementar os métodos: `equals`, que compara os atributos da classe Pessoa para determinar se dois objetos são iguais; e `hashCode`, que retorna um valor para determinar a posição do objeto na coleção. **Use a IDE para gerar esses métodos.**

Set

Conjunto de elementos que não permite elementos duplicados

```
HashSet<String> conjunto = new HashSet<String>();

conjunto.add("A");
conjunto.add("C");
conjunto.add("B");

// Verificando se conseguiu adicionar um elemento
if (conjunto.add("A")){
    System.out.println("Adicionado");
} else {
    System.out.println("Não adicionado");
}

System.out.println(conjunto);

// Removendo todos elementos
conjunto.clear();
```

Queue

Armazena elementos em uma fila

```
Queue<String> fila = new LinkedList<>();

fila.add("Juca");
fila.add("Ana");
fila.add("Maria");

// Remove o primeiro elemento da fila. Retorna null se a fila estiver vazia.
String nome = fila.poll();
if (nome != null) {
    System.out.println(nome);
}
```

- Queue é uma interface que estende Collection, algumas implementações:
 - LinkedList
 - ArrayDeque
 - PriorityQueue

Map

Mapeia chaves para valores (*key, value*)

```
// chave: CPF, valor: Pessoa
HashMap<String, Pessoa> pessoas = new HashMap<>();

// Adicionando elementos
pessoas.put("123", new Pessoa("Juca", 35));
pessoas.put("456", new Pessoa("Maria", 20));
pessoas.put("789", new Pessoa("Ana", 31));

// Verificando se a coleção contém um elemento com chave "123"
Pessoa p = pessoas.get("123");
if (p != null) {
    System.out.println(p);
}

// Imprimindo todos os elementos da coleção
pessoas.forEach((cpf, pessoa) -> {
    System.out.println("CPF: " + cpf);
    System.out.println("Nome: " + pessoa.getNome());
    System.out.println("Idade: " + pessoa.getIdade());
});
```

Map

Usando Streams para manipular elementos

```
// Exibir todos os elementos da coleção que atendem a uma condição, idade > 30
pessoas.values().stream()
    .filter(pessoa -> pessoa.getIdade() > 30)
    .forEach(System.out::println);

// ordenando os elementos da coleção por idade
pessoas.values().stream()
    .sorted(Comparator.comparing(Pessoa::getIdade))
    .forEach(System.out::println);

// Aumentando a idade de todas as pessoas em 1 para quem tem idade maior que 30
pessoas.values().stream()
    .filter(pessoa -> pessoa.getIdade() > 30)
    .forEach(pessoa -> pessoa.setIdade(pessoa.getIdade() + 1));

// Contando quantas pessoas tem idade maior que 30
long count = pessoas.values().stream()
    .filter(pessoa -> pessoa.getIdade() > 30)
    .count();
System.out.println("Total de pessoas com idade maior que 30: " + count);
```

Map

Removendo elementos

```
// Remoção por chave
// Removendo uma pessoa com o CPF "123". Neste caso, a chave é uma String
pessoas.remove("123");

// Não é possível remover diretamente por valor, mas podemos usar removeIf
// para remover elementos que atendam a uma condição
// Exemplo: Remover todas as pessoas com o nome "Ana"
pessoas.values().removeIf(pessoa -> pessoa.getNome().equals("Ana"));

// Se a chave for um objeto da classe Pessoa e não uma String, o método remove() irá
// comparar o objeto passado como parâmetro com os objetos armazenados na coleção,
// utilizando o método equals da classe Pessoa para verificar se são iguais
pessoas.remove(new Pessoa("123", 35));
```

Nota

A classe Pessoa deve implementar os métodos: `equals`, que compara os atributos da classe Pessoa para determinar se dois objetos são iguais; e `hashCode`, que retorna um valor para determinar a posição do objeto na coleção. **Use a IDE para gerar esses métodos.**

Algumas boas práticas com coleções I

- Utilize classes empacotadoras (`Integer`, `Double`, etc.) para tipos primitivos
 - Coleções não aceitam tipos primitivos, apenas objetos
- Utilize `isEmpty()` ao invés de `size() == 0`
- Utilize `contains()` ao invés de `indexOf()`
- Utilize `add()` ao invés de `add(index, elemento)`
- Utilize `removeIf()` ao invés de `remove()`
 - `removeIf()` aceita expressões lambda
- Não use loop `for` com índice para percorrer uma coleção, utilize `for-each` ou `forEach` com lambda

Algumas boas práticas com coleções II

```
ArrayList<Integer> lista = new ArrayList<>();  
lista.add(2);  
lista.add(4);  
lista.add(1);  
  
// Não faça isso  
for (int i = 0; i < lista.size(); i++){  
    System.out.println(lista.get(i));  
}  
  
// Faça isso  
for (String elemento: lista)  
    System.out.println(elemento);  
  
// ou faça isso com lambda  
lista.forEach(elemento -> System.out.println(elemento));
```

Algumas boas práticas com coleções

Quando aprendermos sobre herança e polimorfismo, veremos o porquê

- Utilize a interface mais genérica possível
 - Programar para interfaces, não para implementações deixa o código mais flexível
- Objetos armazenados em coleções devem implementar `equals`, `hashCode` e `Comparable` (se necessário)
 - `equals` para comparar objetos e `hashCode` para calcular o código de espalhamento
 - A interface `Comparable` é usada para ordenar objetos

Material de apoio

- Java Streams: A Revolução na Manipulação de Coleções de dados
 - <https://medium.com/@fabio.alvaro/java-streams-uso-de-streams-api-introduzidas-no-java-8-para-operacoes-funcionais-0d920c26e3c5>